

DELFT UNIVERSITY OF TECHNOLOGY

Private Set Intersection on Intervals

Honours Program Research Proposal

Alexandru Nicolae Lolea

Supervisor: Dr. E. A. Markatou
Department of Cybersecurity

November 19, 2025

1 Introduction

Collaboration has always been the key to rapid and successful technology development in all fields. However, exposing unnecessary information to the party with which you are working increases the risk that malicious individuals might get ahold of that data.

For example, let there be a private hospital and the government of the country in which the hospital is. Now let's say the hospital wants to check which of their patients have already been vaccinated. In order to perform this statistic, they need to compute the intersection between the list of patients that they have and the national vaccination registry. Here comes the security related part: the hospital must protect its patient's data, meaning it can't let the government know which patients it has, while the government also needs to protect the data inside the national vaccination registry.

This is where the Private Set Intersection technique comes into play: Both parties "blind" the elements in their sets and then compute the intersection on that "blinded" data. This is a unifying principle across all PSI protocols, as it guarantees that no additional information is revealed to either side.

2 Research Questions

RQ 1. What are some of the use cases for an "interval-based" PSI algorithm?

RQ 2. What is the time complexity of such algorithms compared to their "single-item" implementations?

RQ 3. Is it feasible to deploy aforementioned algorithms in real world scenarios?

3 Related Work

Private Set Intersection is a very well researched domain, proven by the fact that only in this year has already seen at least 600 publications (verified easily by querying research databases such as ACM Digital Library). Throughout this extensive research, implementations of PSI have two methods of being split. The first method is concerned with the types of parties that take part into the computation itself, which separates PSI into Traditional (only the parties holding the sets participate) and Delegated (blinded sets are sent to a third server that aids in computing the intersection) PSI. The second method is concerned with how strict result matching is. In this regard, the two categories are Classical (matching is done based on strict equality) and Fuzzy PSI (matching is done based on an exact match plus values that are within a specified similarity threshold). In the rest of this section I will present what characterizes each of these categories and give relevant examples of algorithms.

3.1 Traditional PSI

Traditional PSI is defined by the fact that the only parties involved in the computation of intersections are the client and the server. The usual steps taken in order to compute the intersection are the following:

1. Each of the two parties "blind" their sets of data. This can be done through methods such as encryption or hashing
2. The client receives the blinded elements of the server and vice versa
3. Finally, the client computes the intersection (e.g. by raising the keys received from the server to the power of the client's key)

Some examples for this type of PSI include: Diffie-Hellman, De Cristofaro-Tsudik and KKRT PSI.

3.2 Delegated PSI

As mentioned in the beginning of this section, Delegated PSI involves a third party to which computation and/or storage of sets is delegated. This category can be once again split into the following two sub-categories:

1. Algorithms that support one-off delegation
2. Algorithms that support repeated delegation

One-off delegation requires the parties to reencode and send their data to the server for every computation.

Repeated delegation allows the parties to only upload their data once and then re-use it for every computation.

Some Delegated PSI algorithms are the following: Two-Server OPRF-Based Delegated PSI and FSS-Based Delegated PSI

3.3 Classical PSI

Switching the classification axis, we are now categorizing the algorithms based on how strict the matching criteria is. Thus, we begin with Classical PSI which only takes into account exact matches when computing the intersection. Coming back to our original hospital example from the introduction, the Classical PSI algorithms will only output patient IDs that are in both the hospital and national vaccination registries. It is clear why we can't afford to allow room for uncertainty in this case, since we don't want an unvaccinated patient who has a similar ID to one that is vaccinated show up in the computed intersection.

The following examples are Classical PSI algorithms: Standard Deffie-Hellman and Pinkas-Schneider-Zohner PSI

3.4 Fuzzy PSI

By contrast, Fuzzy PSI algorithms compute the intersection of the two sets by computing an exact match and then allowing results that are within a similarity threshold pass as being "close enough". One use case of this class of PSI algorithms is with regards to biometrics. Whenever you create a facial ID, the machine learning algorithm behind it maps your features to values (e.g. depending on skin color, face shape, eye distance etc.). However, it can't always look for an exact match whenever the user wants to access the biometrically secured device may be in a variety of environments which affect values for the features mentioned before (e.g. environmental lighting differs). Thus, it starts from the exact match and looks for similar results.

Examples of these algorithms include Approx-PSI and the Bloom filter PSI for genomic privacy

4 Methodology

In order to conduct the research properly and obtain relevant results in a timely manner, I propose the following methodology: With the help of my supervisor I will first define the stakeholders. By doing this, we will establish what are the main fields that would benefit from the PSI extension that will be developed. Afterwards I will begin by implementing the standard version of PSI algorithms from each of the categories mentioned above and then try to adapt them to work with sets of intervals. For every implementation I will run metrics in order to gather data related to the speed before (using the standard way) and after extension.

Finally, I will pick the use case with the most improvement as the deliverable of this project.

5 Timeline

The following timeline will go up to the month of May. I chose to write it two months before the end of my studies since possible obstacles must be taken into account. Given the scale of the research project, I believe this amount of time is sufficient to overcome any such barrier and still deliver a working, tested and documented product.

1. November + first week of December - implement the standard Diffie-Hellman PSI algorithm;
2. December - implement Traditional PSI algorithms
3. January - implement Delegated PSI algorithms
4. February - implement Classical PSI algorithms

5. March - implement Fuzzy PSI algorithms and decide together with the supervisor on a concrete interval as the deliverable
6. April - implement the chosen algorithm
7. June - connect metrics and perform statistics which will be used in the report
8. May - writing and revising of report

6 References